การหาจุดตัด (Choke Point) ของเส้นทางโจมตีบนโครงสร้างพื้นฐานแบบ Hybrid: วิธีให้น้ำหนักที่อธิบายที่มาได้ และการวัดว่าผลลัพธ์เชื่อถือได้แค่ไหน

Independent Study Project • 06016421 INFORMATION TECHNOLOGY INFRASTRUCTURE SECURITY • ภาคเรียนที่ 1/2569

สมาชิกและหน้าที่: นายธนบดี อังกุลดี รหัสนักศึกษา 67070065 — ทำคนเดียวทั้งหมด: (i) ออกแบบโครงสร้างกราฟและเขียนตัวสร้างกราฟจำลอง; (ii) จัด lab ทั้งฝั่ง network และ AWS แล้วดึงข้อมูลเข้า Neo4j; (iii) ทำวิธีให้น้ำหนัก edge และเขียนตัววิเคราะห์; (iv) ออกแบบการทดลอง วิเคราะห์ผลเชิงสถิติ และเขียนรายงาน

บทคัดย่อ

องค์กรทุกวันนี้มีทั้งเครือข่ายภายใน (on-premise) และระบบบนคลาวด์ ผู้โจมตีที่ยึดเครื่องแรกได้มักไม่หยุดแค่นั้น แต่จะไต่ต่อไปทีละขั้นจนถึงข้อมูลสำคัญ เครื่องมือกลุ่ม Attack Path Management เช่น BloodHound, Wiz และ XM Cyber แก้ปัญหานี้ด้วยการมองทั้งระบบเป็นกราฟ แล้วหาเส้นทางที่ผู้โจมตีเดินได้ ซึ่งทำได้ดีอยู่แล้ว แต่ยังเหลือช่องว่างสองจุดที่ตระหนักได้

ปัญหาแรกคือความน่าเชื่อถือของน้ำหนักบนแต่ละเส้นทาง. แม้เครื่องมือปัจจุบันจะมีการให้คะแนนเพื่อจัดอันดับว่าเส้นทางใดมีความอันตรายที่สุด แต่คะแนนดังกล่าวมักบอกเพียงว่าใช้ปัจจัยใดบ้างในการคำนวณ ไม่สามารถอธิบายที่มาเชิงคณิตศาสตร์ได้อย่างชัดเจน และไม่เคยมีใครรายงานว่าถ้าคะแนนคลาดเคลื่อนไปสัก 20–30% อันดับที่ได้จะเปลี่ยนหรือไม่ **ปัญหาจุดต่อมาคือขาดความสามารถในการวิเคราะห์ข้ามโดเมน.** เครื่องมือฝั่ง network (RedSeal, Skybox) กับฝั่ง cloud (Wiz) เป็นคนละตัวกัน edge ที่ข้ามระหว่างสองโครงสร้างจึงกลายเป็นจุดบอดของทั้งคู่ เช่น ไฟล์ `~/.aws/credentials` ที่ตกค้างบนเซิร์ฟเวอร์ในองค์กร หรือการใช้ SSRF ดึง token ออกจาก IMDS ทั้งที่การโจมตีจริงมักเดินผ่านทางนี้

โครงงานนี้แก้ทั้งสองจุดด้วยการวิเคราะห์ **สองชั้นบนกราฟเดียวกัน** ชั้นแรกใช้ **dominator tree** หาจุดที่ทุกเส้นทางต้องผ่าน ตัดตรงนั้นแล้วเส้นทางขาดแน่นอน โดยไม่ต้องพึ่งตัวเลขน้ำหนักเลย ชั้นที่สองใส่น้ำหนัก $w(e) = -\log p(e)$ ซึ่งทำให้การหา shortest path ด้วย Dijkstra มีความหมายทางคณิตศาสตร์เป็นเส้นทางที่ผู้โจมตีน่าจะใช้มากที่สุดจริงๆ ไม่ใช่การเดา โดยค่า $p(e)$ ของ edge ที่มี CVE ดึงจาก EPSS ได้ตรงๆ ส่วน edge ที่เป็น misconfig ซึ่งไม่มี CVE ให้ดู จะประมาณค่าจากฟังก์ชันที่ fit ไว้ระหว่างเวกเตอร์ CVSS กับ EPSS บนข้อมูล CVE จริง จึงได้ค่าที่อยู่บนสเกลเดียวกัน และชี้แจงที่มาได้ทุกตัว **ผลลัพธ์หลักของโครงงานนี้คือ ผลการวัดและอันดับ Choke point** โดยทดสอบบนกราฟจำลองหลายร้อยกราฟว่าอันดับ choke point ทนต่อการรบกวนน้ำหนักได้แค่ไหน ทั้งนี้แนวคิด attack graph มีมาตั้งแต่ Sheyner ในปี 2002 และ MulVAL ปี 2005; ส่วนที่โครงงานนี้เพิ่มเข้าไปคือวิธีให้น้ำหนักที่ตรวจสอบได้ การวัดความน่าเชื่อถือของผลลัพธ์ และการรวมสองโดเมนเข้าด้วยกัน

วัตถุประสงค์

1. ออกแบบกราฟการโจมตีที่ใช้ “สถานะของผู้โจมตี” เป็น node คือคู่ของทรัพยากรกับระดับสิทธิ์ (เช่น เครื่อง A ระดับ root) และครอบคลุม edge ที่ข้ามจาก network ไป cloud อย่างชัดเจน
2. ทำวิธีให้ค่า $p(e)$ ที่อธิบายที่มาได้ทุกตัว ทั้ง edge ที่มี CVE (ใช้ EPSS) และ edge ที่เป็น misconfig (เทียบค่าจากความสัมพันธ์ CVSS–EPSS)
3. ใช้ dominator tree หา choke point ที่รับประกันได้บนกราฟมีทิศทาง แล้วเทียบกับผลจาก weighted min-cut เพื่อดูว่าสองวิธีให้คำตอบตรงกันหรือไม่
4. วัดว่าอันดับ choke point เปลี่ยนแค่ไหนเมื่อรบกวนค่าน้ำหนักและเมื่อข้อมูลกราฟขาดหาย รวมถึงตอบว่าการเพิ่ม EPSS ช่วยได้จริงกว่าการใช้ CVSS เฉยๆ หรือไม่
5. สร้าง lab จริงทั้งสองฝั่งด้วย GNS3 + Nessus และ AWS + Terraform แล้วดึงข้อมูลเข้า Neo4j ด้วย Cartography เพื่อยืนยันว่าโมเดลใช้กับระบบจริงได้

ขอบเขต

อยู่ในขอบเขต:

- กราฟจำลองอย่างน้อย 200 กราฟ สำหรับงานสถิติ + กราฟจริงจาก lab ทั้ง network, cloud และแบบรวมสองฝั่ง
- การวิเคราะห์เชิงโครงสร้าง (dominator tree) และเชิงความน่าจะเป็น ($-\log p$) บนกราฟชุดเดียวกัน
- การทดลองความทนทาน: รบกวนน้ำหนัก $\pm$10–50%, สุ่มลบ edge 5–20%, และเปรียบเทียบโมเดลน้ำหนัก 4 แบบ
- ข้อสมมติที่ประกาศไว้ล่วงหน้า: ใช้ OR-semantics (ไม่รองรับ AND-node แบบ MulVAL), สมมติว่าผู้โจมตีไม่เสียสิทธิ์ที่ได้มาแล้ว และสมมติว่า edge เป็นอิสระต่อกัน

นอกขอบเขต: การสั่งแก้ไขการตั้งค่าโดยอัตโนมัติ; การเขียนตัวเก็บข้อมูลขึ้นใหม่แทน Cartography; การตรวจจับแบบ real-time; และการรองรับ AND-node เต็มรูปแบบ

ผลที่คาดว่าจะได้รับ

- วิธีให้น้ำหนัก edge ที่ชี้แจงที่มาได้ทุกค่า พร้อมเกณฑ์เทียบค่าที่คนอื่นทำซ้ำได้
- ตัวเลขว่าน้ำหนักคลาดเคลื่อนได้กี่เปอร์เซ็นต์ก่อนอันดับ choke point จะเชื่อถือไม่ได้ (รายงานด้วย Kendall's τ และสัดส่วนการซ้อนทับของอันดับต้น)
- อัตราที่ dominator tree กับ weighted min-cut ให้คำตอบไม่ตรงกัน ซึ่งเป็นข้อมูลที่เครื่องมือปัจจุบันไม่รายงาน
- คำตอบว่าต้องแบ่ง segment เครือข่ายละเอียดแค่ไหน ถึงจะเกิดจุดตัดที่รับประกันได้จริง และสัดส่วนของเส้นทางอันตรายที่วิ่งผ่าน edge ข้ามโดเมน
- Artifact: ตัววิเคราะห์และตัวสร้างกราฟจำลองแบบ open source พร้อมชุดข้อมูลผลการทดลองและรายงานฉบับสมบูรณ์

เอกสารอ้างอิงหลัก. [1] O. Sheyner et al., "Automated Generation and Analysis of Attack Graphs," *IEEE S&P*, 2002. [2] X. Ou et al., "MulVAL: A Logic-based Network Security Analyzer," *USENIX Security*, 2005. [3] S. Noel et al., "Efficient Minimum-Cost Network Hardening via Exploit Dependency Graphs," *ACSAC*, 2003. [4] T. Lengauer and R. Tarjan, "A Fast Algorithm for Finding Dominators in a Flowgraph," *ACM TOPLAS*, 1979. [5] FIRST, "Exploit Prediction Scoring System (EPSS)."